

Relatório A2 – DijkFood

Computação em Nuvem

Elisa Soares Maria Eduarda Mesquita
Mariana Fernandes Raphaella Roma

June 2026

1 Arquitetura

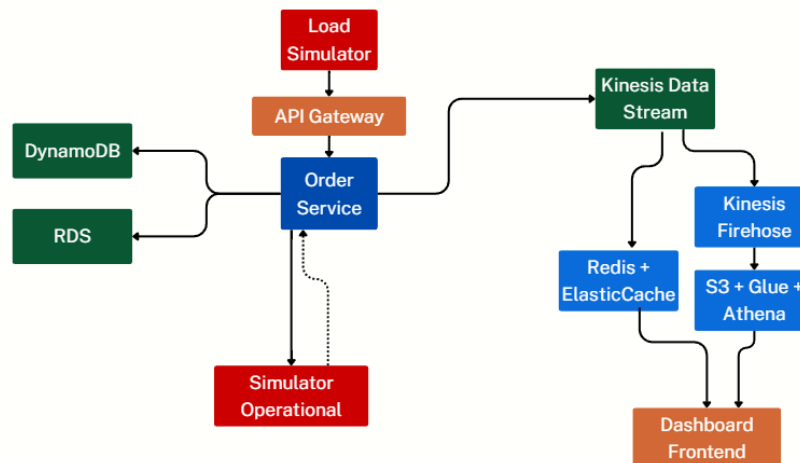


Figure 1: Arquitetura geral do DijkFood

O fluxo começa no **Load Simulator**, responsável por gerar carga sintética para o sistema. Ele envia requisições para a API por meio do **API Gateway**, que representa o ponto de entrada do sistema. A partir desse ponto, as chamadas chegam ao **Order Service**, serviço central da aplicação. Esse serviço coordena a criação dos pedidos, consulta e atualiza dados persistentes e publica eventos de mudança de estado.

Na camada de persistência, o sistema utiliza dois tipos de armazenamento. O **RDS** armazena dados relacionais da aplicação, como pedidos, usuários, restaurantes e entidades transacionais. O **DynamoDB** é usado para dados que se beneficiam de acesso rápido e escalabilidade horizontal, como localização ou

estado operacional de entregadores. Essa separação permite usar banco relacional onde há relacionamento forte entre entidades e banco NoSQL onde há necessidade de baixa latência e alto volume de leitura/escrita.

O **Simulador Operacional** representa o comportamento externo do ambiente de delivery. Ele simula etapas como aceitação do pedido, preparo, despacho e entrega. Na arquitetura, ele se comunica com o Order Service para gerar mudanças de estado e alimentar o ciclo de vida dos pedidos. Assim, o sistema consegue produzir eventos semelhantes aos de uma operação real, mesmo sem usuários humanos interagindo continuamente.

Sempre que eventos relevantes ocorrem, o Order Service publica registros no **Kinesis Data Stream**. Esse componente funciona como a espinha dorsal de eventos da arquitetura. Ele desacopla a geração dos eventos do consumo posterior, permitindo que diferentes consumidores leiam o mesmo fluxo de dados sem depender diretamente do Order Service.

A partir do Kinesis, existem dois caminhos principais de observação. O primeiro é o caminho de **tempo real**, no qual os eventos são consumidos por serviços de métricas e podem ser propagados para o **Redis/ElastiCache**. No ambiente local, esse componente é um Redis em container; na AWS, ele é representado pelo **Amazon ElastiCache for Redis**. Ele pode atuar tanto como banco de snapshots quanto como mecanismo de Pub/Sub, permitindo que o dashboard receba atualizações com baixa latência, mas aqui, como fluxo final decidimos por pub-sub (o que será defendido em uma seção posterior).

O segundo caminho é o caminho **analítico**. Nele, os eventos do Kinesis são enviados para o **Kinesis Firehose**, que grava os dados no **S3**. Em seguida, o **Glue** organiza o catálogo dos dados e o **Athena** permite consultas SQL sobre os eventos armazenados. Esse fluxo não tem como objetivo atualizar a interface em tempo real, mas sim permitir análises históricas, agregações por período e relatórios posteriores.

Por fim, o **Dashboard Frontend** consome tanto os dados de tempo real quanto os dados analíticos. Para a visão operacional, ele recebe snapshots via WebSocket a partir dos serviços de métricas e do Redis/ElastiCache. Para a visão histórica, ele consulta resultados agregados vindos da trilha S3, Glue e Athena. Dessa forma, a arquitetura separa claramente o que precisa de baixa latência do que pode ser processado de forma assíncrona e analítica.

2 Gargalos

2.1 Gargalo no Routing Service

Durante o desenvolvimento, a integração do *courier-simulator* com o *routing-service* introduziu um gargalo não imediatamente óbvio. Após a correção da rota do entregador, que passou a seguir o caminho calculado pelo algoritmo de Dijkstra em vez de uma interpolação direta entre dois pontos, o sistema apresentou um comportamento indesejado: havia muitos pedidos no estado *ready to pickup* e entregadores disponíveis, mas apenas um ou dois entregadores proces-

savam entregas simultaneamente.

A causa foi identificada na forma como o *delivery-service* solicitava rotas ao *routing-service*: todas as requisições eram processadas por uma única *task*, criando uma fila serializada. Os entregadores existiam e estavam disponíveis, mas ficavam bloqueados aguardando a resposta de rota antes de iniciar o deslocamento. O gargalo não estava na alocação nem na quantidade de entregadores, mas na capacidade de processamento paralelo das requisições de roteamento.

A solução foi o aumento do número de *tasks* paralelas dedicadas ao *routing-service*, permitindo que múltiplas requisições de rota fossem processadas simultaneamente. Adicionalmente, o serviço foi colocado atrás de um balanceador de carga, permitindo distribuir as requisições entre múltiplas instâncias, e o *autoscaling* foi reconfigurado com base no volume de *requests*. Com isso, a capacidade do *routing-service* passa a crescer conforme a demanda observada, reduzindo o risco de ele se tornar o ponto único de contenção no fluxo de entregas.

3 Dashboard

3.1 Fluxo em tempo real

A origem comum dos quatro caminhos é o *Kinesis Data Stream*, que recebe eventos do ciclo de vida dos pedidos, como criação, mudança de status, atribuição de entregador e conclusão da entrega. A partir desse ponto, os caminhos são independentes entre si: cada um consome ou propaga os dados de uma forma diferente e publica snapshots para o dashboard por WebSocket.

O primeiro caminho é o **Realtime Metrics Service sem PySpark**. Nesse fluxo, o serviço FastAPI possui um consumidor direto do Kinesis implementado com `boto3`. O consumidor percorre os shards do stream, desserializa cada registro, identifica o formato do evento e atualiza um estado agregado em memória. Esse estado mantém contadores de pedidos em preparo, aguardando entregador, em entrega, entregues, entregadores disponíveis e latências de ingestão. A cada segundo, o backend envia o snapshot diretamente ao dashboard por WebSocket. Esse caminho tem a menor complexidade operacional, pois não depende de uma camada intermediária além do próprio serviço de métricas.

O segundo caminho usa **Redis como banco de snapshots**. Um worker separado também consome o Kinesis em paralelo, calcula as mesmas métricas agregadas e grava o resultado no Redis com uma chave de snapshot e tempo de expiração. O backend do dashboard consulta periodicamente esse snapshot no Redis e repassa o conteúdo para o navegador por WebSocket. Nesse desenho, o Redis funciona como uma camada de estado compartilhado: ele desacopla o cálculo das métricas da entrega para o dashboard e permite que múltiplas instâncias do backend leiam o mesmo snapshot. A desvantagem é que existe uma etapa extra de escrita e leitura, o que tende a adicionar latência e a exigir cuidado com TTL, consistência do snapshot e frequência de atualização.

O terceiro caminho usa **Redis como Pub/Sub**. Nesse fluxo, o worker

que consome o Kinesis não apenas grava o snapshot no Redis, mas também publica a mensagem em um canal Redis Pub/Sub. O backend do dashboard fica inscrito nesse canal e, ao receber cada publicação, encaminha o snapshot imediatamente para os clientes WebSocket. A principal diferença em relação ao Redis como banco é o padrão de comunicação: em vez de consultar uma chave periodicamente, o backend recebe uma notificação por push. Isso reduz o custo de polling e tende a ser melhor para baixa latência, embora mensagens Pub/Sub sejam efêmeras: se um consumidor estiver desconectado, ele não recebe publicações antigas.

O quarto caminho usa **PySpark para métricas em tempo real**. Esse pipeline também consome eventos do Kinesis, mas processa uma janela recente de eventos usando uma sessão PySpark local. O objetivo é avaliar se uma ferramenta de processamento distribuído traria vantagem para agregações mais pesadas. Nesse caso, os eventos são convertidos para estruturas compatíveis com Spark, agregados por janela e transformados em snapshots enviados ao dashboard. A vantagem desse caminho aparece quando o volume de dados e a complexidade das transformações justificam um motor distribuído. Entretanto, para o dashboard operacional existe um custo adicional de inicialização, serialização e execução do Spark, o que pode impactar o desempenho em cargas maiores.

Para comparar os caminhos, foi executada uma carga local controlada com 5 pedidos por segundo durante 60 segundos, totalizando 300 tentativas de pedido. O sistema aceitou 246 pedidos e concluiu 71 entregas durante a execução observada. A métrica principal da comparação foi `event_to_dashboard_emit_latency_ms`, que mede o intervalo entre o timestamp do evento original e o momento em que o backend emite o snapshot para o dashboard. Portanto, os valores refletem o tempo fim a fim entre o evento publicado no fluxo e a emissão da atualização visual, e não apenas o tempo de processamento interno de cada componente.

Nas tabelas, a coluna **amostras** representa a quantidade de medições de latência coletadas para cada caminho a partir dos snapshots emitidos pelo backend; ela não representa diretamente a quantidade de pedidos criados. A coluna **média** representa a latência média dessas amostras. A coluna **p50** representa o percentil 50, ou seja, metade das medições teve latência menor ou igual àquele valor. A coluna **p95** representa o percentil 95, isto é, 95

A Tabela 1 mostra o comportamento em um cenário controlado. Nesse cenário, o caminho com **PySpark** apresentou os melhores resultados absolutos, com menor média, menor p50 e menor p95 entre todos os caminhos, sendo o único a atingir p95 inferior a 500 ms. Os caminhos baseados em Redis ficaram muito próximos entre si, enquanto o caminho sem PySpark apresentou latência mais elevada nesse cenário específico.

Além da medição executada no cenário controlado, também foram executados cenários adicionais de carga. O cenário *teste* usou 1 pedido por segundo durante 1 segundo, apenas para validar o funcionamento dos caminhos. O cenário *normal* usou 5 pedidos por segundo durante 60 segundos. O cenário *pico* usou 50 pedidos por segundo durante 60 segundos. O cenário *especial* usou 200 pedidos por segundo durante 60 segundos, forçando uma carga muito acima do uso

normal. A Tabela 2 organiza os dados por cenário e por caminho.

A leitura dos cenários mostra que o comportamento dos caminhos varia conforme a intensidade da carga. No cenário *normal*, o caminho de **Métricas sem PySpark** apresentou a menor latência média, com 455.60 ms, embora seu p95 ainda tenha ficado acima de 500 ms. Nesse mesmo cenário, o Redis Pub/Sub apresentou latência média superior, mas manteve distribuição mais estável do que o Redis como banco.

No cenário *pico*, todos os caminhos sofreram aumento significativo de latência devido ao acúmulo de backlog gerado pela saturação da API local antes do dashboard. Mesmo assim, o caminho com **Redis Pub/Sub** apresentou o melhor equilíbrio entre latência média, desacoplamento e estabilidade operacional, superando os demais sob carga mais intensa.

Os resultados mostram que não existe um único caminho dominante em todos os cenários. O PySpark foi superior em ambiente controlado de baixa carga, mostrando boa eficiência para agregações em janelas pequenas. O caminho sem PySpark teve melhor desempenho no cenário operacional normal, enquanto o Redis Pub/Sub apresentou maior resiliência em cenários de maior volume. Isso mostra que o comportamento depende não apenas do algoritmo de agregação, mas também da forma de propagação do estado até o dashboard.

Considerando não apenas latência absoluta, mas também escalabilidade, desacoplamento entre componentes e robustez sob carga, o caminho escolhido para o projeto foi o **Redis como Pub/Sub**. Embora não tenha sido o melhor em todos os cenários, foi o que apresentou comportamento mais consistente à medida que a carga aumentou. Além disso, seu modelo baseado em push elimina polling constante, reduz acoplamento entre backend e worker e facilita escalar múltiplas instâncias do dashboard. O Redis como banco permanece útil como fallback para consulta do último snapshot, enquanto o caminho sem PySpark continua competitivo em cenários simples. O PySpark, por sua vez, fica mais adequado para pipelines analíticos ou transformações mais pesadas, e não como caminho principal do dashboard operacional.

3.2 Limitação no requisito de latência

Um dos requisitos não funcionais definidos para o dashboard operacional era atingir latência de p95 inferior a 500 ms no fluxo de atualização em tempo real. Durante os experimentos, diferentes abordagens foram testadas, incluindo consumo direto do Kinesis, Redis como banco de snapshots, Redis Pub/Sub e processamento com PySpark.

Ao longo do desenvolvimento, foram realizadas diversas otimizações, como redução de polling, desacoplamento entre workers e backend, uso de comunicação por push via Pub/Sub, aumento de paralelismo em serviços internos e ajustes na frequência de emissão de snapshots. Apesar dessas tentativas, o requisito de $p95 < 500$ ms não foi consistentemente atingido nos cenários operacionais mais representativos.

Embora o PySpark tenha conseguido atingir esse objetivo em um cenário controlado de baixa carga, esse comportamento não se manteve sob carga normal

ou pico. Assim, conclui-se que, dentro das limitações de infraestrutura local e da complexidade do sistema, o requisito não funcional não foi plenamente satisfeito, ainda que a arquitetura tenha sido ajustada para se aproximar desse objetivo.

3.3 Fluxo analítico

O fluxo analítico foi implementado como a trilha assíncrona da arquitetura, separada do caminho de baixa latência usado pelo dashboard operacional. Enquanto o fluxo em tempo real precisa transformar eventos em snapshots quase imediatamente, o fluxo analítico prioriza persistência, consulta histórica e agregações posteriores. Essa separação evita que consultas mais pesadas ou relatórios sobre períodos maiores concorram com o caminho responsável por atualizar a interface em tempo real.

A origem dos dados continua sendo o **Kinesis Data Stream**. O *Order Service* publica eventos do ciclo de vida dos pedidos, incluindo identificador do pedido, tipo do evento, status anterior, status novo, mensagem do evento, coordenadas e timestamp de criação. A partir do stream, o **Kinesis Firehose** atua como componente de entrega para armazenamento durável. Ele consome os registros do Kinesis e grava os dados brutos no **S3**, usando uma organização particionada por **year**, **month**, **day** e **hour**. Essa partição reduz o volume lido em consultas por data e torna o armazenamento mais adequado para análises históricas.

Sobre os arquivos armazenados no S3, o **Glue Data Catalog** define uma tabela externa para os eventos de pedidos. A tabela usa formato JSON e registra os campos principais dos eventos, como **event_id**, **order_id**, **event_type**, **from_status**, **to_status**, **event_status**, **event_message**, **latitude**, **longitude** e **created_at**. No deploy, também foi configurada projeção de partições, permitindo que o Athena interprete a estrutura de diretórios no S3 sem depender de uma etapa manual de cadastro de cada partição criada.

O **Athena** é a camada de consulta do fluxo analítico. Ele executa SQL diretamente sobre a tabela externa do Glue e permite responder perguntas que dependem de dados acumulados, como quantidade total de eventos por dia, distribuição de pedidos por status, eventos por tipo e volume por hora. No dashboard, o endpoint **/metrics/analytics** consulta o Athena e retorna um snapshot analítico com indicadores como total de eventos, total de pedidos, pedidos abertos, pedidos entregues, pedidos cancelados, taxa de entrega, taxa de cancelamento, distribuição por status, distribuição por tipo de evento e série de eventos por hora.

Uma diferença importante em relação ao fluxo em tempo real é que o fluxo analítico não tem compromisso com atualização instantânea. O Firehose trabalha com buffer antes de gravar no S3 e o Athena consulta dados já persistidos. Por isso, essa trilha é mais adequada para relatórios, auditoria e visões consolidadas, enquanto o Redis Pub/Sub permanece como caminho escolhido para a experiência operacional do dashboard. Na arquitetura final, os dois fluxos se complementam: o caminho em tempo real mostra o estado recente da operação, e o caminho analítico preserva o histórico para consultas e avaliações posteriores.